

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de

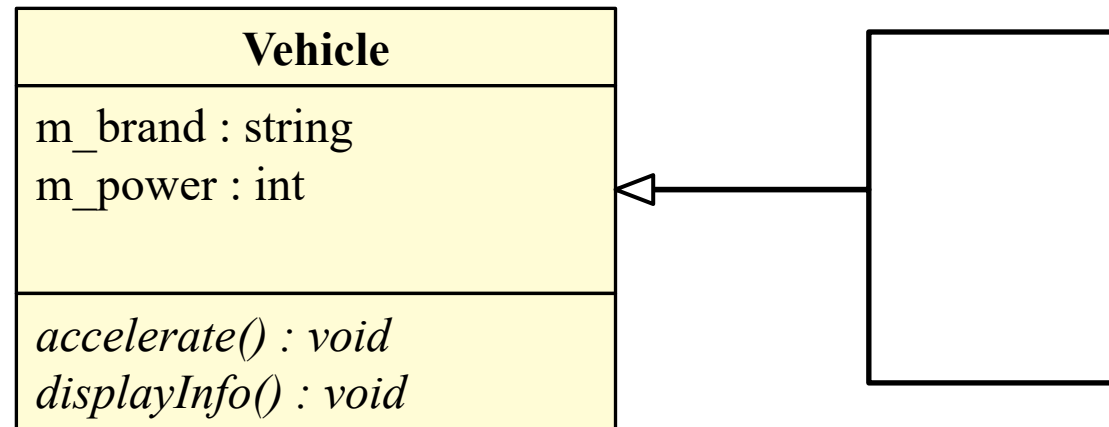


Recap

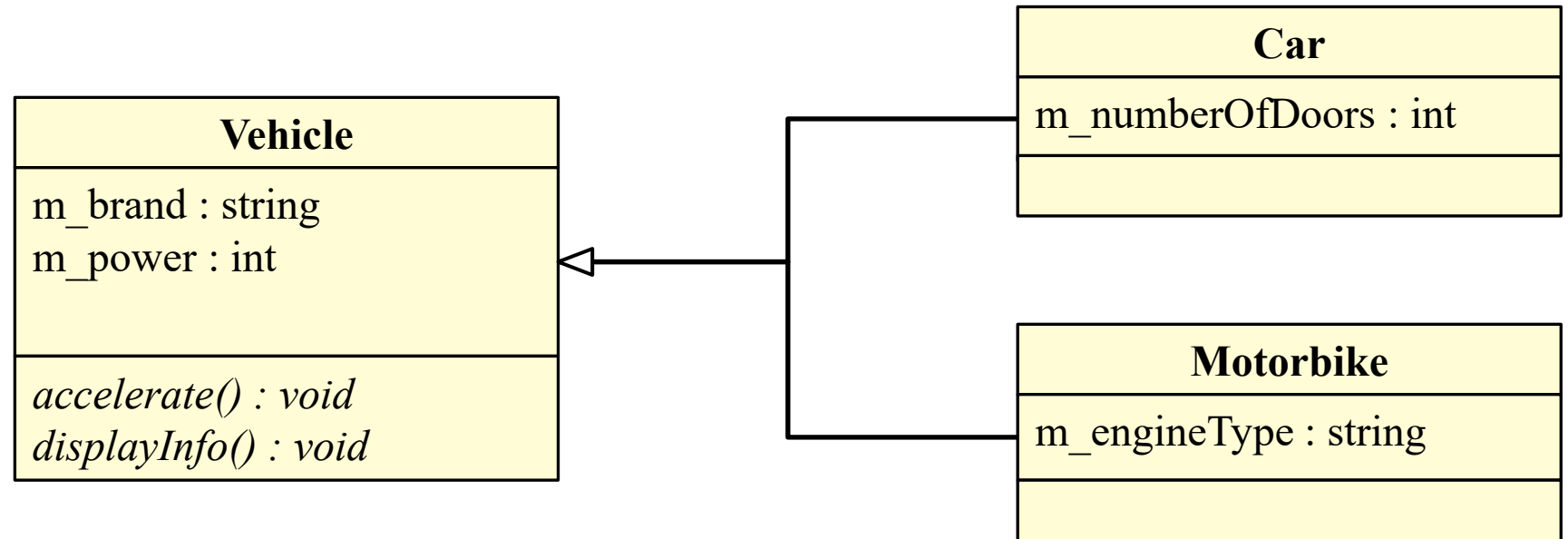
- Vorstellungen Aufgaben
- Q&A Dokumente, um wieder in die Themen reinzukommen
 - OOP_Cpp_Part_4_Q&A
 - OOP_Cpp_Part_5_Q&A
 - keywords_demo

Recap Vererbung & UML

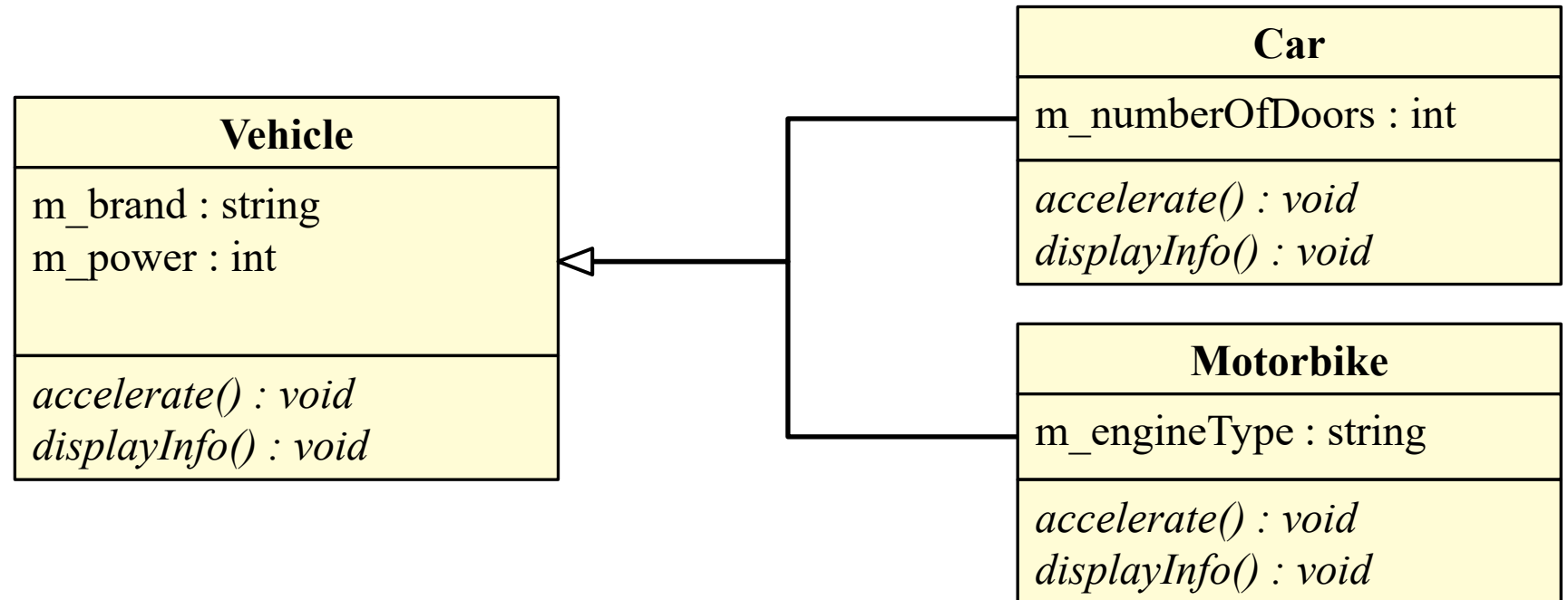
Recap Vererbung & UML



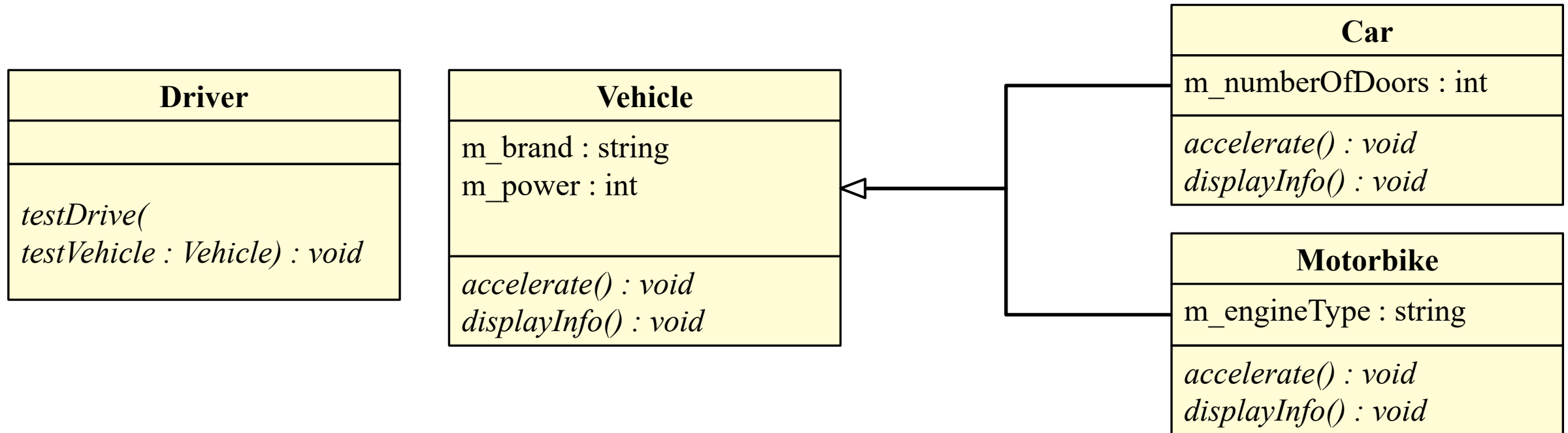
Recap Vererbung & UML



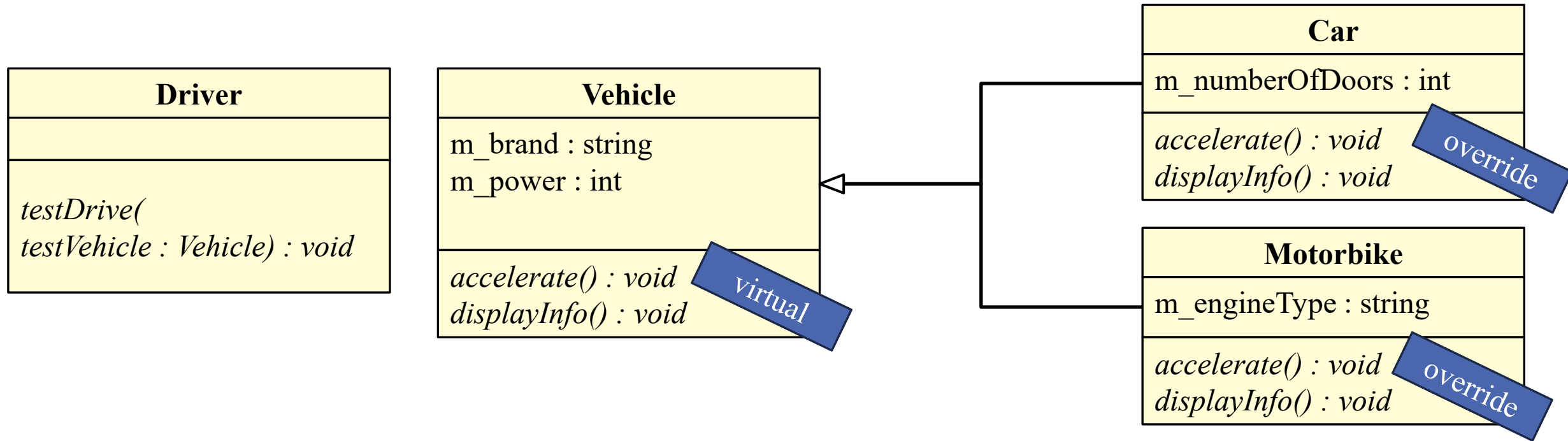
Recap Vererbung & UML



Recap Vererbung & UML



Recap Vererbung, Polymorphie & UML



Aufgaben: Recap

- 1) Implementieren sie das vorangegangene Beispiel mit den Klassen `Vehicle`, `Car`, `Motorbike` polymorph.
 - Alle Variablen werden im Konstruktor gesetzt.
 - Die Funktion `displayInfo` gibt `brand`, `power` und `numberOfDoors/engineType` aus
 - Die Funktion `accelerate` gibt den String „accelerate with power“ und `m_power` aus. Beim `Motorbike` wird außerdem ausgegeben, dass ein Wheely stattfindet, beim `Car`, dass ein Drift stattfindet.
- 2) Erstellen Sie die Klasse `Driver`
 - Der `Driver` hat eine Funktion `testVehicle`, welches ein `Vehicle` übergeben bekommt
 - Die Funktion gibt die Eigenschaften und das Beschleunigungsverhalten von `Vehicle` aus.
 - Nutzen Sie Referenzen/Pointer zur Übergabe und zur Ermöglichung von Polymorphie.
- 3) Ihre `main` soll einen ein objekt `Driver`, sowie ein `Car` und ein `Motorbike` erstellen. Der `Driver` soll dann beide Fahrzeuge mit `testVehicle` testen.

Implementieren Sie alle Klassen in `.hpp` und `.cpp` Files!

Kapitel 3: Vertiefung C++

31. Weitere C++ Keywords

32. Namespaces



33. Objektorientiertes Design

Am Beispiel des Single Responsibility Principle

34. Übungsprojekt

Single Responsibility Principle (SRP)

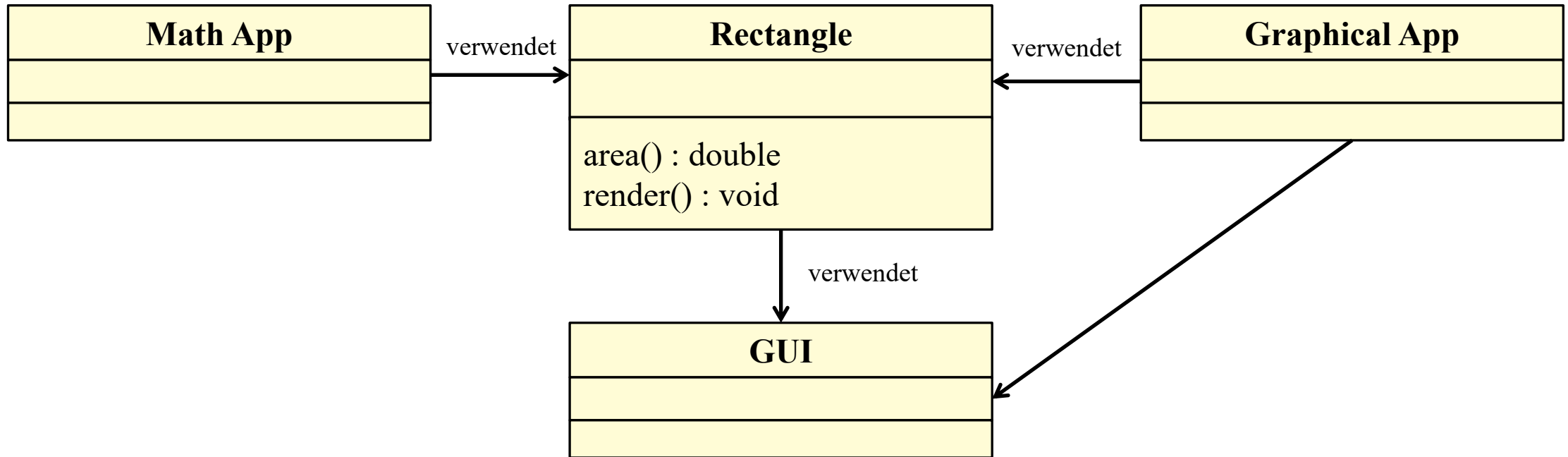
Jede Klasse sollte für genau einen Teil der Funktionalität eines Programms verantwortlich sein.

Diese Funktionalität sollte vollständig in der Klasse gekapselt sein.

- Damit sollte es nur einen einzigen Grund geben warum eine Klasse geändert werden muss.

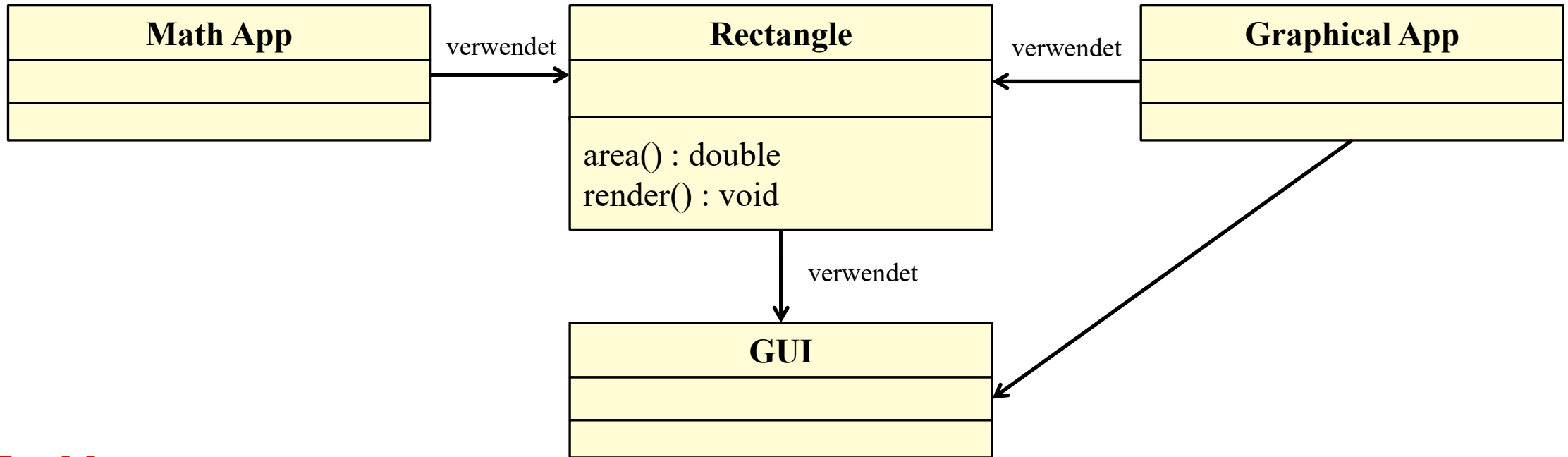
SRP: Die Gott-Klassen

Bad example!



SRP: Die Gott-Klassen

Bad example!

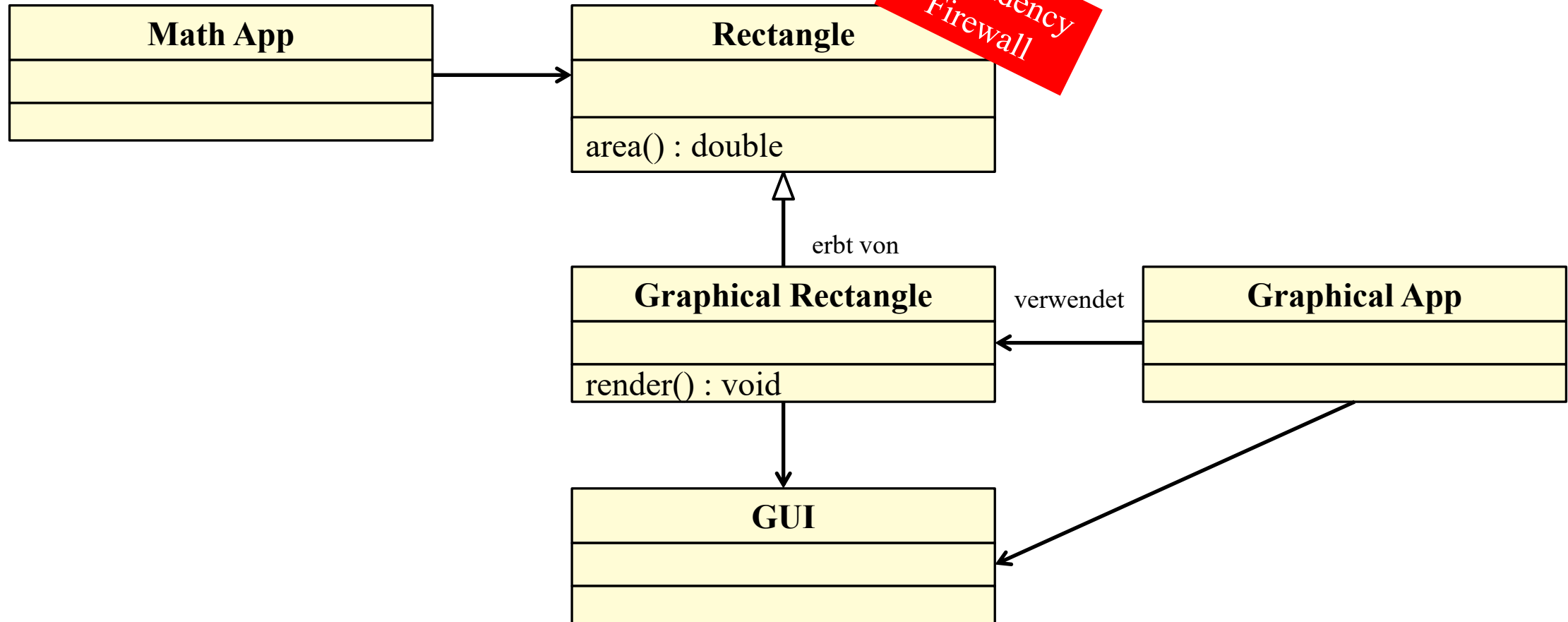


Probleme

- **Rectangle** hat mehr als eine Verantwortung: Grafische Darstellung + mathematische Berechnung
- Math App ist abhängig von GUI (diese müssen für Math App inkludiert werden)
- Änderungen in der GUI können zu Fehlern in Math App führen

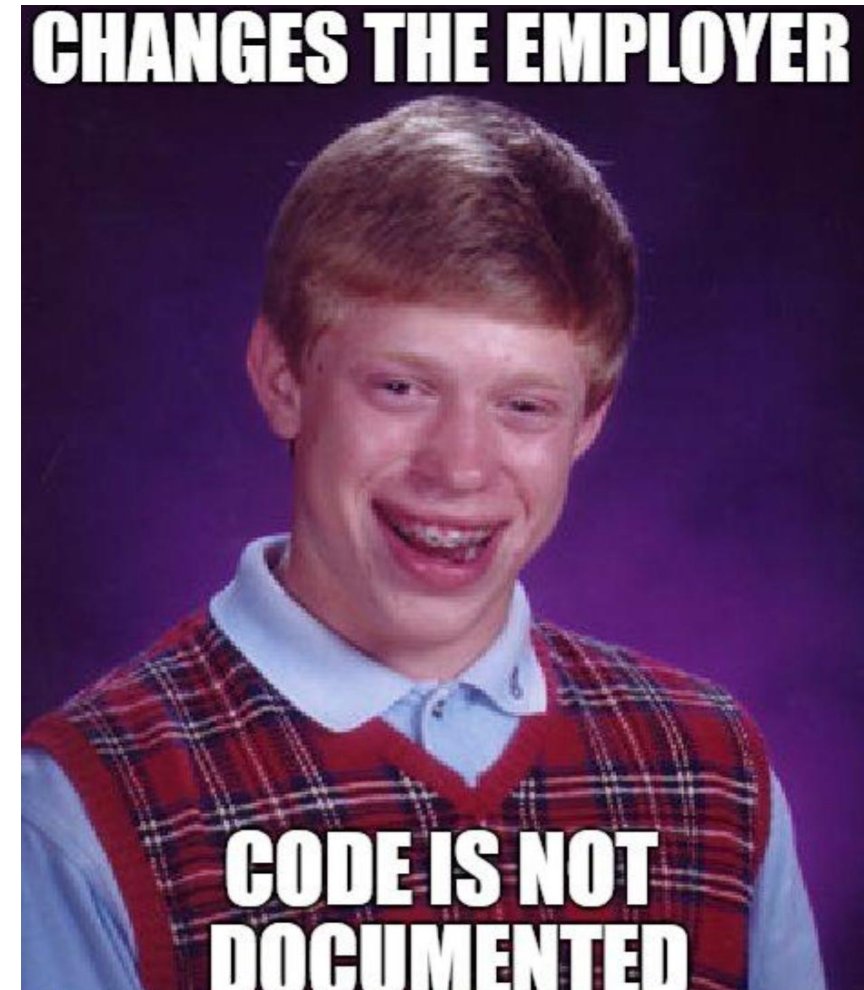
Single Responsibility Principle (SRP)

Good example!



Beide Rectangle Klassen sind nur für **eine** Funktionalität verantwortlich

Design – Warum?



Weitere Designprinzipien (SOLID)

- **Single Responsibility Principle**: Jede Klasse nur eine Funktion
 - **Open/Closed Principle**: Offen für Erweiterung, geschlossen für Veränderung
 - **Liskov Substitution Principle**: Abgeleitete Klassen sollen sich wie die Basisklasse verhalten lassen.
 - **Interface Segregation Principle**: Verwende mehrere kleine spezialisierte Schnittstellen
 - **Dependency Inversion Principle**: Abhängigkeit von Abstraktionen, nicht von konkreten Klassen.
 - Siehe SOLID_Principles.md in GIT
- [SOLID Principles in Programming: Understand With Real Life Examples - GeeksforGeeks](#)

Weitere Vorteile von OOP Design

- **Abstraktion**: Können Dinge gleich behandelt werden? Dann sollte das auch passieren über Interfaces/Vererbung oder Wiederverwendung
- **Nähe zur Realität**: Je näher etwas der Realität ist, desto weniger muss man umdenken. Viel leichtere Kommunikation mit Kunde/Management
- **Divide & Conquer**: Je kleiner Funktionen und Klassen, desto einfacher wartbar, verständlich und erweiterbar
- **Kapselung**: Je weniger Abhängigkeiten, desto besser! Klassen so schneiden, dass Abhängigkeiten reduziert werden (einer der größten Vorteile gegenüber prozedural)

Exkurs: CMake

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)**
- 5 Clean Code
- 6 Test-driven Development

Kapitel 4: Die Standard Template Library

- 41. Container
- 42. Iteratoren
- 43. Algorithmen
- 44. Templates

Einführung IBANGen

Einführung IBANGen

```
//search for used IBAN using binary search
bool IBANGen::isUsed(int IBAN) {
    int low = 0, high = (int) IBANGen::usedVectors.size();
    int mid = (low + high) / 2;
    while (low <= high && !IBANGen::usedVectors.empty()) {
        if (IBAN < usedVectors[mid]) {
            high = --mid;
        } else if (IBAN > usedVectors[mid]) {
            low = ++mid;
        } else if (IBAN == usedVectors[mid]) {
            return true;
        }
        mid = (low + high) / 2;
    }
    return false;
}

void bubbleSort(std::vector<int> &v, int start, int end) {
    bool swapped = true;
    for (int i = start; i < end && swapped; ++i) {
        swapped = false;
        for (int j = start; j < end-1-i; ++j) {
            if (v[j+1] < v[j]) {
                std::swap(v[j], v[j+1]);
                swapped = true;
            }
        }
    }
}

//add iban to used vectors and sort it
void IBANGen::add(int IBAN) {
    IBANGen::usedVectors.push_back(IBAN);
    bubbleSort(IBANGen::usedVectors, 0, IBANGen::usedVectors.size());
}
```

```
//add iban to used vectors and sort it
void IBANGen::add(int IBAN) {
    IBANGen::usedVectors.push_back(IBAN);
    bubbleSort(IBANGen::usedVectors, 0, IBANGen::usedVectors.size());
}

//search for IBAN using STL
bool IBANGen::isUsedSTL(int IBAN) {
    //find searches for the element in the vector and returns an iterator t
    return std::find(IBANGen::usedVectors.begin(), IBANGen::usedVectors.end(), IBAN) != IBANGen::usedVectors.end();
}

//add iban to used vectors and sort it with STL
void IBANGen::addSTL(int IBAN) {
    IBANGen::usedVectors.push_back(IBAN);
    std::sort(IBANGen::usedVectors.begin(), IBANGen::usedVectors.end());
}
```

Standard Template Library - STL

- Sie wollen ein Problem lösen: Vielleicht hat es schon jemand anderes gelöst? Vielleicht sogar ...
 - ... **schneller**
 - ... **effizienter**: RAM und ROM schonender
 - ... besser **getestet**
 - ... besser **portierbar**
 - ... dazu **sparen** Sie **Zeit**!
 - Bibliotheken sollten immer in Betracht gezogen werden!
 - **C++ bietet z.B. die STL**
- Elemente hieraus befinden sich im `std:: namespace`

Weitere Gründe für die STL

Besser lesbarer Code

- *Raising level of abstraction*

Fehler vermeiden

- z.B. *Off-by-one*

Wird von (fast) allen Entwicklern benutzt

- Geteiltes Vokabular
- Mit (fast) allen Compilern und C++ Versionen gleich verfügbar

Ausgewählte Header der STL

<code><algorithm></code>	<code>copy(),find(),sort()</code>
<code><array></code>	<code>array</code>
<code><chrono></code>	<code>duration,time_point</code>
<code><cmath></code>	<code>sqrt(),pow()</code>
<code><forward_list></code>	<code>forward_list</code>
<code><iostream></code>	<code>cout,cin,ostream,istream</code>
<code><map></code>	<code>map,multimap</code>
<code><memory></code>	<code>unique_ptr,shared_ptr</code>
<code><random></code>	<code>normal_distribution</code>
<code><regex></code>	<code>regex</code>
<code><string></code>	<code>string</code>
<code><set></code>	<code>set,multiset</code>
<code><utility></code>	<code>move(),swap(),pair</code>
<code><vector></code>	<code>vector</code>

→ [Header Finden: C++ Standard Library headers - cppreference.com](http://cppreference.com)